

MongoDB Live Coding API

Purpose

This project demonstrates how to build a small Express API connected to MongoDB with Mongoose. It uses two related resources: categories and products.

The live coding covers:

- Express application setup
- MongoDB connection with Mongoose
- Two Mongoose models
- An ObjectId relationship between products and categories
- Schema validation
- POST and GET endpoints
- Product filters with query parameters
- populate for related category data
- JSON error responses
- Testing with Postman or Insomnia

1. Requirements

- Node.js
- npm
- MongoDB running locally or a MongoDB connection URI

2. Project structure

```
mongodb-live-coding/  
|-- app.js  
|-- routes.js  
|-- package.json  
|-- .env  
|-- src/  
|   |-- controller/  
|   |   |-- category.controller.js  
|   |   |-- product.controller.js  
|   |-- modules/  
|   |   |-- category.module.js  
|   |   |-- product.module.js  
|   |-- routes/  
|       |-- category.routes.js  
|       |-- product.routes.js
```

3. Installation

Install the dependencies:

```
npm install
```

Create a .env file:

```
MONGO_URI=mongodb://127.0.0.1:27017/live_mongoose  
PORT=3000
```

Start the development server:

```
npm run dev
```

The API is available at <http://localhost:3000>.

4. Application setup

The application loads environment variables, enables JSON request bodies, logs requests with Morgan, mounts the API routes, and connects to MongoDB.

app.js

```
import "dotenv/config"
import express from "express";
import mongoose from "mongoose";
import routes from './routes.js'
import morgan from "morgan";

const app = express();
app.use(express.json());
app.use(morgan('dev'))
app.use("/api" , routes)

try {
  await mongoose.connect(process.env.MONGO_URI)
} catch (error) {
  console.error(error.message)
}

app.listen(process.env.PORT, () => {
  console.log(`backend is running on PORT ${process.env.PORT}`)
})
```

5. Category model

A category has a required unique name and a required description. Mongoose also adds createdAt and updatedAt through timestamps.

src/modules/category.module.js

```
import mongoose from "mongoose";

const categoryschema = new mongoose.Schema({
  name: {
    type: String,
    required: true,
    trim: true,
    minlength: 2,
    unique: true,
  },
  description: {
    type: String,
    trim: true,
    maxlength: 200,
    required: true
  },
}, { timestamps: true })

const Category = mongoose.model('Category', categoryschema)

export default Category
```

6. Product model and relationship

A product stores its own name and price. The category field stores an ObjectId that references a Category document. This is a reference rather than embedded category data.

src/modules/product.module.js

```
import mongoose from "mongoose";

const productSchema = new mongoose.Schema({
  name: {
    type: String,
    required: true,
    trim: true,
    minlength: 2
  },
  price: {
    type: Number,
    required: true,
    min: 0
  },
  category: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'Category',
    required: true
  }
}, { timestamps: true })

const Product = mongoose.model('Product', productSchema)

export default Product
```

7. Category controller

The category controller creates categories and returns all categories. Validation errors are transformed into JSON responses.

src/controller/category.controller.js

```

import Category from "../modules/category.module.js";

export async function getCategories(req, res) {
  try {
    const categories = await Category.find()

    res.status(200).json({
      success: true,
      data: {
        categories
      }
    })
  } catch (error) {
    res.status(400).json({
      success: false,
      error: error.message
    })
  }
}

export async function StoreCategories(req, res) {
  try {
    const category = await Category.create(req.body)

    res.status(201).json({
      success: true,
      data: {
        category
      }
    })
  } catch (error) {
    res.status(400).json({
      success: false,
      error: error.message
    })
  }
}

```

8. Product controller, filters, and populate

The product list reads query parameters and builds a MongoDB filter. The result is enriched with the related category using populate.

Supported filters:

- name: case-insensitive partial name search
- category: exact category ObjectId
- minPrice: minimum price
- maxPrice: maximum price

src/controller/product.controller.js

```

import Product from "../modules/product.module.js";

export async function getProducts(req, res) {
  try {
    const { name, category, minPrice, maxPrice } = req.query;

    const filter = {};

    if (name) {
      filter.name = { $regex: name, $options: "i" };
    }

    if (category) {
      filter.category = category;
    }

    if (minPrice || maxPrice) {
      filter.price = {};

      if (minPrice) filter.price.$gte = Number(minPrice);
      if (maxPrice) filter.price.$lte = Number(maxPrice);
    }

    const products = await Product.find(filter)
      .populate("category");

    res.status(200).json({
      success: true,
      message: 'products loaded successfully',
      data: {
        products
      }
    });
  } catch (error) {
    res.status(500).json({
      success: false,
      message: "Failed to fetch products",
      error: error.message
    });
  }
}

export async function StoreProducts(req, res) {
  try {
    const product = await Product.create(req.body)

    res.status(201).json({
      success: true,
      message: "product created successfully",
      data: {
        product
      }
    })
  } catch (error) {
    res.status(400).json({
      success: false,
      message: "Failed to create product",
      error: error.message
    })
  }
}

```

9. Routes

routes.js

```
import express from "express"
import categoryRoutes from './src/routes/category.routes.js'
import productRoutes from './src/routes/product.routes.js'

const router = express.Router()

router.use('/categories', categoryRoutes)
router.use('/products', productRoutes)

export default router
```

src/routes/category.routes.js

```
import express from "express"
import { StoreCategories, getCategories } from '../controller/category.controller.js';

const router = express.Router();
router.get('/', getCategories)
router.post('/', StoreCategories)

export default router
```

src/routes/product.routes.js

```
import express from 'express'
import { getProducts, StoreProducts } from '../controller/product.controller.js'

const router = express.Router();
router.get('/', getProducts)
router.post('/', StoreProducts)

export default router
```

10. Testing with Postman or Insomnia

Create a category

POST http://localhost:3000/api/categories
Content-Type: application/json

```
{
  "name": "Electronics",
  "description": "Electronic products"
}
```

Save the returned category _id.

Create a product

POST http://localhost:3000/api/products
Content-Type: application/json

```
{
  "name": "Laptop",
  "price": 1200,
  "category": "CATEGORY_ID"
}
```

Replace CATEGORY_ID with the real category _id.

List products with the related category

```
GET http://localhost:3000/api/products
```

Filter products

```
GET http://localhost:3000/api/products?name=laptop
GET http://localhost:3000/api/products?category=CATEGORY_ID
GET http://localhost:3000/api/products?minPrice=500&maxPrice=1500
```

Example successful response

```
{
  "success": true,
  "message": "products loaded successfully",
  "data": {
    "products": [
      {
        "name": "Laptop",
        "price": 1200,
        "category": {
          "name": "Electronics",
          "description": "Electronic products"
        }
      }
    ]
  }
}
```

Example validation error

Sending a product without a required field returns a JSON error response:

```
{
  "success": false,
  "message": "Failed to create product",
  "error": "Product validation failed"
}
```

11. Project status and remaining improvement

The project currently implements the main live coding requirements: two related Mongoose models, ObjectId references, schema validation, MongoDB connection, POST routes, filtered GET products, populate, and JSON error responses.

One recommended improvement remains: before creating a product, explicitly check that the referenced category exists with `Category.findById(req.body.category)`. Mongoose validates the ObjectId format, but it does not automatically guarantee that the referenced document exists.

Other optional improvements are numeric query validation, a global 404 handler, automated tests, and fuller README documentation.

12. Key concepts to explain

- A directly stored value is saved inside the current MongoDB document.
- A reference stores another document's ObjectId.
- Mongoose validation checks incoming document values against a schema.
- A query filter limits which documents MongoDB returns.

- populate replaces a referenced ObjectId with related document data in the response.